

Juan Esteban Correa

Emmanuel Berrio Jimenez

Medellín

INSTITUCION UNIVERSITARIA

PASCUAL BRAVO

1 OCTUBRE 2026

INFORME

Evaluación práctica de la Unidad 1: Reconociendo lo aprendido

Miembros:

Emmanuel Berrio Jimenez

Juan Esteban Correa

Repositorio:

[Emmanuelxs13/Tarea BigData Unidad 01](#)

Caso de Estudio: Empresa “Gaseosas Poderosas”

1. Descripción de la Tarea

El presente trabajo consiste en el diseño, depuración y análisis de una base de datos para la empresa "Gaseosas Poderosas". Las actividades principales abarcan la creación de un modelo conceptual (Diagrama de Chen) y un modelo lógico. Posteriormente, se lleva a cabo un proceso ETL (Extracción, Transformación y Carga) en el cual se incorpora una nueva dimensión espacial (Regiones), seguido de una exhaustiva limpieza e imputación de datos anómalos en la tabla de transacciones. Finalmente, a través de una vista consolidada (vista_operaciones), se formulan consultas SQL para extraer indicadores clave de rendimiento (KPIs), los cuales son presentados mediante gráficos de Pareto y de torta para facilitar la toma de decisiones gerenciales.

Actividades Realizadas:

A) DISEÑO CONCEPTUAL Y LÓGICO DE LA BASE DE DATOS

Se diseñó un modelo entidad-relación que incluye:

- Tabla 'departamentos': Almacena información de departamentos de Colombia
- Tabla 'municipios': Contiene datos de municipios vinculados a departamentos
- Tabla 'productos': Registro de productos comercializados
- Tabla 'operaciones': Registro transaccional de ventas
- Tabla 'temporal': Tabla de apoyo para transformaciones

B) PROCESO ETL (EXTRACCIÓN, TRANSFORMACIÓN, CARGA)

EXTRACCIÓN:

- Archivo CSV del DANE con departamentos y municipios de Colombia
- Script SQL con registros de operaciones de ventas
- Algoritmo Python para generación aleatoria de datos

TRANSFORMACIÓN:

- Conversión de códigos DANE a identificadores numéricos únicos
- Normalización de formato de fechas a 'AAAA-MM-DD'
- Adición de campo 'id_región' para clasificación geográfica
- Limpieza de datos con problemas de integridad

CARGA:

- Base de datos PostgreSQL 'bigdata'
- Tablas normalizadas con integridad referencial

C) LIMPIEZA DE DATOS

Se implementó un proceso exhaustivo de validación y corrección de:

- Registros con fechas en formato incorrecto
- Valores nulos o cero en cantidad de ventas
- Valores negativos resultantes de errores de ingreso
- Códigos de departamento faltantes

D) ANÁLISIS Y CONSULTAS

Se realizaron consultas SQL complejas utilizando:

- Operaciones JOIN para combinar múltiples tablas
- Funciones de agregación (SUM, COUNT, AVG)
- Cláusulas GROUP BY y ORDER BY para clasificación
- DISTINCT para eliminación de duplicados

E) VISUALIZACIÓN

Se generaron gráficos de Pareto que permiten:

- Identificar los 20% de variables que generan el 80% de resultados
- Tomar decisiones informadas basadas en datos
- Visualizar tendencias de ventas por región

2. Diagrama de Entidad -Relación (Diagrama de Chen)

ESTRUCTURA DEL MODELO ENTIDAD-RELACIÓN

El siguiente diagrama representa las entidades, atributos y relaciones de la base de datos:

ENTIDAD	ATRIBUTOS CLAVE	RELACIONES
DEPARTAMENTOS	• id_departamento (PK) • nombre • abb • codigo_dane • codigo_region • población	1:N con MUNICIPIOS 1:N con OPERACIONES
MUNICIPIOS	• id_municipio (PK) • id_departamento (FK) • nombre • abb • codigo_dane • población	N:1 con DEPARTAMENTOS 1:N con OPERACIONES
PRODUCTOS	• id_producto (PK) • nombre • precio	1:N con OPERACIONES
OPERACIONES	• id_registro (PK) • id_departamento (FK) • id_municipio (FK) • id_producto (FK) • fecha • cantidad • estado	N:1 con DEPARTAMENTOS N:1 con MUNICIPIOS N:1 con PRODUCTOS

Características del Modelo:

- NORMALIZACIÓN: Las tablas están normalizadas hasta la 3ª forma normal (3FN)
- INTEGRIDAD REFERENCIAL: Se mantienen restricciones de clave foránea
- ESCALABILIDAD: Diseño preparado para crecer sin rediseño
- PERFORMANCE: Índices en claves primarias y foráneas para optimizar consultas

3. Diccionario de Datos

TABLA: departamentos

Columna	Tipo	Nulabilidad	Descripción
id_departamento	INTEGER	NOT NULL	Identificador único del departamento (PK)
nombre	VARCHAR(70)	NOT NULL	Nombre completo del departamento
abb	VARCHAR(3)	NULL	Abreviatura de 3 letras
codigo_dane	VARCHAR(10)	NULL	Código DANE del departamento
codigo_region	INTEGER	NULL	Identificador de región geográfica
población	INTEGER	NULL	Número de habitantes

TABLA: municipios

Columna	Tipo	Nulabilidad	Descripción
id_departamento	INTEGER	NOT NULL	Clave foránea a departamentos (FK)
id_municipio	INTEGER	NOT NULL	Identificador único del municipio (PK)
nombre	VARCHAR(70)	NOT NULL	Nombre del municipio
codigo_dane	VARCHAR(10)	NULL	Código DANE del municipio

TABLA: productos

Columna	Tipo	Nulabilidad	Descripción
id_producto	INTEGER	NOT NULL	Identificador único (PK)
nombre	VARCHAR(20)	NOT NULL	Nombre comercial
precio	INTEGER	NULL	Precio unitario

TABLA: operaciones

Columna	Tipo	Nulabilidad	Descripción
id_registro	INTEGER	NOT NULL	Identificador único (PK)
id_departamento	INTEGER	NOT NULL	Clave foránea (FK)
id_municipio	INTEGER	NOT NULL	Clave foránea (FK)
id_producto	INTEGER	NOT NULL	Clave foránea (FK)
fecha	VARCHAR(10)	NULL	AAAA-MM-DD
cantidad	INTEGER	NULL	Unidades vendidas
estado	VARCHAR(1)	NULL	V=válido, I=inválido

4. Corrida del algoritmo ETL

FASE 1: EXTRACCIÓN (Obtención de los datos)

Paso 1.1: Cargar el Archivo CSV del DANE (Fuente 1)

```
1 COPY temporal(region, codigo_dane_departamento, departamento, codigo_dane_municipio, municipio)
2 FROM 'C:/Users/Public/colombia-dane-departamentos.csv'
3 WITH (
4     FORMAT csv,
5     HEADER true,
6     DELIMITER ',',
7     QUOTE '"',
8     ESCAPE '\',
9     ENCODING 'UTF8'
10 );
```

Data Output Messages Notifications

COPY 1123

Query returned successfully in 63 msec.

Query Query History

```
1 SELECT 'operaciones' AS tabla, COUNT(*) AS total_registros FROM operaciones
2 UNION ALL
3 SELECT 'productos' AS tabla, COUNT(*) AS total_registros FROM productos;
```

Data Output Messages Notifications

	tabla text	total_registros bigint
1	operacion...	10000
2	productos	4

FASE 2: TRANSFORMACIÓN

```
1 -- 2.1 Conversión de Códigos DANE (Elimina puntos en códigos como '5.001')
2 UPDATE temporal
3 SET codigo_dane_departamento = REPLACE(codigo_dane_departamento, '.', ''),
4     codigo_dane_municipio = REPLACE(codigo_dane_municipio, '.', '');
5
6
```

Data Output Messages Notifications

UPDATE 1123

Query returned successfully in 79 msec.

QueryQuery History

1-- 2.2 Normalización de Fechas (Soporta YYYY-MM-DD, DD-MM-YYYY y MM-DD-YYYY)

2UPDATE operaciones

3SET fecha = CASE

4-- Caso 1: El año está al inicio (ej: '2024-8-21')

5WHEN length(split_part(fecha, '-', 1)) = 4 THEN

6TO_CHAR(TO_DATE(fecha, 'YYYY-MM-DD'), 'YYYY-MM-DD')

7

8-- Caso 2: El año está al final (ej: '21-08-2024' o '11-23-2024')

9WHEN length(split_part(fecha, '-', 3)) = 4 THEN

10CASE

11-- Si el segundo valor es mayor a 12, es formato MM-DD-YYYY ('11-23-2024')

12WHEN split_part(fecha, '-', 2)::integer > 12 THEN

13TO_CHAR(TO_DATE(fecha, 'MM-DD-YYYY'), 'YYYY-MM-DD')

14-- En caso contrario, es formato DD-MM-YYYY ('21-08-2024')

15ELSE

16TO_CHAR(TO_DATE(fecha, 'DD-MM-YYYY'), 'YYYY-MM-DD')

17END

18

19ELSE fecha

20END

21WHERE fecha LIKE '%-%-%';

Data OutputMessagesNotifications

UPDATE 10000

Query returned successfully in 102 msec.

1-- 2.3 Adición de columna id_region

2ALTER TABLE operaciones ADD COLUMN IF NOT EXISTS id_region INTEGER;

Data OutputMessagesNotifications

ALTER TABLE

Query returned successfully in 56 msec.

FASE 3: CARGA

Query	Query History
1	-- Paso 3.1: Cargar Departamentos
2	INSERT INTO departamentos (id_departamento, nombre, codigo_dane)
3	SELECT
4	ROW_NUMBER() OVER (ORDER BY departamento) AS id_departamento,
5	departamento AS nombre,
6	codigo_dane_departamento AS codigo_dane
7	FROM (
8	SELECT DISTINCT departamento, codigo_dane_departamento
9	FROM temporal
10	WHERE departamento IS NOT NULL
11	) deptos_unicos;

Data Output	Messages	Notifications
INSERT 0 33		
Query returned successfully in 57 msec.		

Query	Query History
1	-- 3. Paso 3.2: Cargar Municipios
2	INSERT INTO municipios (id_municipio, id_departamento, nombre, codigo_dane)
3	SELECT
4	ROW_NUMBER() OVER (ORDER BY t.municipio) AS id_municipio,
5	d.id_departamento,
6	t.municipio AS nombre,
7	t.codigo_dane_municipio AS codigo_dane
8	FROM temporal t
9	JOIN departamentos d ON t.codigo_dane_departamento = d.codigo_dane
10	WHERE t.municipio IS NOT NULL;

Data Output	Messages	Notifications
INSERT 0 1123		
Query returned successfully in 112 msec.		

Query

Query History

1

2

3

4

5

6

7

SELECT 'departamentos' AS tabla, COUNT(*) AS total_registros FROM departamentos

UNION ALL

SELECT 'municipios' AS tabla, COUNT(*) AS total_registros FROM municipios

UNION ALL

SELECT 'productos' AS tabla, COUNT(*) AS total_registros FROM productos

UNION ALL

SELECT 'operaciones' AS tabla, COUNT(*) AS total_registros FROM operaciones;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

	tabla text	total_registros bigint
1	departament...	33
2	municipios	1123
3	productos	4
4	operaciones	10000

5. Modificar el algoritmo ETL para incluir dato de región.

Paso 5.1: Crear la tabla regiones e insertar los datos, Crearemos la tabla regiones y le asignaremos un ID único secuencial (id_region) a cada región geográfica encontrada en la tabla temporal.

Query	Query History
1	-- 1. Crear la tabla relacional de regiones
2	CREATE TABLE IF NOT EXISTS regiones (
3	id_region INTEGER PRIMARY KEY,
4	descripcion TEXT NOT NULL
5	);
6	
7	-- 2. Poblar la tabla con las regiones únicas del DANE
8	INSERT INTO regiones (id_region, descripcion)
9	SELECT
10	ROW_NUMBER() OVER (ORDER BY region) AS id_region,
11	region AS descripcion
12	FROM (
13	SELECT DISTINCT region
14	FROM temporal
15	WHERE region IS NOT NULL AND region <> ''
16	) r_unicas;

Data Output	Messages	Notifications
INSERT 0 6		
Query returned successfully in 84 msec.		

Paso 5.2: Garantizar la columna id_region en operaciones

Query	Query History
1	ALTER TABLE operaciones ADD COLUMN IF NOT EXISTS id_region INTEGER;

Data Output	Messages	Notifications
NOTICE: la columna «id_region» de la relación «operaciones» ya existe, omitiendo ALTER TABLE		
Query returned successfully in 58 msec.		

Paso 5.3: Valorizar (poblar) id_region en la tabla operaciones

Para asignar la región correcta a cada una de las 10.000 operaciones, cruzamos la información a través de los municipios o departamentos hacia la tabla temporal y regiones.

```
Query  Query History
1  UPDATE operaciones o
2  SET id_region = sub.id_region
3  FROM (
4      SELECT DISTINCT m.id_municipio, r.id_region
5      FROM municipios m
6      JOIN temporal t ON m.codigo_dane = t.codigo_dane_municipio
7      JOIN regiones r ON t.region = r.descripcion
8  ) sub
9  WHERE o.id_municipio = sub.id_municipio;
```

Data Output Messages Notifications

UPDATE 0

Query returned successfully in 67 msec.

```
Query  Query History
1  UPDATE operaciones o
2  SET id_region = sub.id_region
3  FROM (
4      SELECT DISTINCT d.id_departamento, r.id_region
5      FROM departamentos d
6      JOIN temporal t ON d.codigo_dane = t.codigo_dane_departamento
7      JOIN regiones r ON t.region = r.descripcion
8  ) sub
9  WHERE o.id_departamento = sub.id_departamento;
```

Data Output Messages Notifications

UPDATE 0

Query returned successfully in 69 msec.

Paso 5.4: Consulta de Verificación Final

Query

Query History

```
1  SELECT
2      r.id_region,
3      r.descripcion AS nombre_region,
4      COUNT(o.*) AS total_operaciones
5  FROM regiones r
6  LEFT JOIN operaciones o ON r.id_region = o.id_region
7  GROUP BY r.id_region, r.descripcion
8  ORDER BY r.id_region;
```

Data Output

Messages

Notifications

	id_region [PK] integer	nombre_region text	total_operaciones bigint
1	1	Region Caribe	1601
2	2	Region Centro Oriente	3266
3	3	Region Centro Sur	1140
4	4	Region Eje Cafetero - Antioq...	440
5	5	Region Llano	605
6	6	Region Pacifico	1638

6. Detectar registros con problemas. Transformación de datos

Paso 6.1: Crear los campos de auditoría

Query	Query History
1	ALTER TABLE operaciones ADD COLUMN IF NOT EXISTS modificado VARCHAR (2) DEFAULT 'NO';
2	ALTER TABLE operaciones ADD COLUMN IF NOT EXISTS causa VARCHAR (255) DEFAULT 'Válido';

Data Output	Messages	Notifications
ALTER TABLE		
Query returned successfully in 86 msec.		

Paso 6.2: Detectar ejemplos reales de id_registro para el informe

Query	Query History
1	-- Buscar un ID de ejemplo para cada Tipo (A, B, C, D, E)
2	SELECT 'Tipo A (Fecha No ISO)' AS tipo, id_registro, fecha AS valor_detectado FROM operaciones WHERE fecha NOT LIKE '____'
3	UNION ALL
4	SELECT 'Tipo B (Cantidad 0)', id_registro, cantidad::text FROM operaciones WHERE cantidad = 0
5	UNION ALL
6	SELECT 'Tipo C (Cantidad Negativa)', id_registro, cantidad::text FROM operaciones WHERE cantidad < 0
7	UNION ALL
8	SELECT 'Tipo D (Depto en 0)', id_registro, id_departamento::text FROM operaciones WHERE id_departamento = 0 OR id_departamento IS NULL
9	UNION ALL
10	SELECT 'Tipo E (Producto en 0)', id_registro, id_producto::text FROM operaciones WHERE id_producto = 0 OR id_producto IS NULL

Data Output	Messages	Notifications
Showing rows: 1 to 26		
tipo	id_registro	valor_detectado
text	integer	character varying
1	Tipo A (Fecha No ISO)	12110 024-07-17
2	Tipo A (Fecha No ISO)	14368 24-28-10
3	Tipo A (Fecha No ISO)	15955 24-27-09
4	Tipo A (Fecha No ISO)	18056 24-09-09
5	Tipo A (Fecha No ISO)	11599 24-02-08
6	Tipo A (Fecha No ISO)	16558 01-02-24
7	Tipo B (Cantidad 0)	10225 0
8	Tipo B (Cantidad 0)	15412 0
9	Tipo B (Cantidad 0)	17528 0
10	Tipo B (Cantidad 0)	19181 0
11	Tipo B (Cantidad 0)	13161 0
12	Tipo C (Cantidad Negati...	11907 -144
13	Tipo C (Cantidad Negati...	15208 -175
14	Tipo C (Cantidad Negati...	18126 -181
15	Tipo C (Cantidad Negati...	19538 -112
16	Tipo C (Cantidad Negati...	10014 -198
17	Tipo D (Depto en 0)	10326 0
18	Tipo D (Depto en 0)	12188 0
19	Tipo D (Depto en 0)	14020 0

Paso 6.3: Corregir e imputar los datos (Tipos A al E)

IDs de registro de ejemplo:

- **Tipo A:** 12110 (o 14368)
- **Tipo B:** 10225
- **Tipo C:** 11907
- **Tipo D:** 10326

QueryQuery History

```

1  -----
2  -- Tipo C: Cantidades Negativas (Convertir a positivo)
3  -----
4  UPDATE operaciones
5  SET cantidad = ABS(cantidad),
6      modificado = 'SI',
7      causa = 'Signo Negativo Corregido'
8  WHERE cantidad < 0;
9
10 -----
11 -- Tipo D: Falta código de departamento (Imputar desde el Municipio)
12 -----
13 UPDATE operaciones o
14 SET id_departamento = sub.depto_id,
15     modificado = 'SI',
16     causa = 'Departamento Imputado'
17 FROM (
18     SELECT DISTINCT
19         o2.id_municipio,
20         d.id_departamento AS depto_id

```

Data OutputMessagesNotifications

UPDATE 5

Query returned successfully in 77 msec.

Paso 6.4: Consulta de Verificación Final (Evidencia)

QueryQuery History

```

1  SELECT
2      modificado,
3      causa,
4      COUNT(*) AS total_registros
5  FROM operaciones
6  GROUP BY modificado, causa
7  ORDER BY modificado DESC;

```

Data OutputMessagesNotifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	modificado character varying (2)	causa character varying (255)	total_registros bigint
1	SI	Imputación Promedio Muni...	5
2	SI	Signo Negativo Corregido	5
3	SI	Departamento Imputado	1
4	SI	Producto Imputado (Naranji...	5
5	NO	Válido	9984

Tipo de problema	Número Registro	Campo (BD)	Problema detectado	Solución
Tipo A	12110	fecha	Formato diferente a AAAA-MM-DD (024-07-17).	Normalización de cadena con lógica CASE y TO_DATE(). Se actualiza a modificado='SI' y causa='Formato Fecha Corregido'.
Tipo B	10225	cantidad	Cantidad en 0.	Imputación del valor promedio (AVG) de ventas del municipio correspondiente. Se asigna modificado='SI' y causa='Imputación Promedio Municipio'.
Tipo C	11907	cantidad	Cantidad negativa (-144).	Conversión a positivo mediante la función ABS(cantidad). Se asigna modificado='SI' y causa='Signo Negativo Corregido'.
Tipo D	10326	id_departamento	Código de departamento en 0 o nulo.	Imputación cruzando el id_departamento perteneciente al municipio de la operación. Se asigna modificado='SI' y causa='Departamento Imputado'.
Tipo E	10014	id_producto	Código de producto en 0 o nulo en Támesis.	Asignación directa del código del producto "NARANJITA" . Se asigna modificado='SI' y causa='Producto Imputado (Naranjita)'.

7. Tabla de consultas SQL

Consultas SQL de Negocio sobre vista_operaciones

#	Descripción	Consulta SQL
7.1	Seleccionar los 8 departamentos con mayor volumen de ventas (monto) de productos ordenados de mayor a menor.	SELECT departamento, SUM(total_venta) AS monto_total FROM vista_operaciones GROUP BY departamento ORDER BY monto_total DESC LIMIT 8;
7.2	Seleccionar los 15 municipios con mayor cantidad de productos vendidos en el departamento de Antioquia ordenados de mayor a menor.	SELECT municipio, SUM(cantidad) AS cantidad_total FROM vista_operaciones WHERE UPPER(departamento) = 'ANTIOQUIA' GROUP BY municipio ORDER BY cantidad_total DESC LIMIT 15;
7.3	Seleccionar los 5 departamentos con mayor cantidad de gaseosas vendidas del producto "MANZALOCA" ordenados de mayor a menor.	SELECT departamento, SUM(cantidad) AS cantidad_total FROM vista_operaciones WHERE UPPER(producto) LIKE '%MANZALOCA%' GROUP BY departamento ORDER BY cantidad_total DESC LIMIT 5;
7.4	Seleccione los 5 municipios con el menor monto de ventas de gaseosas ordenados de menor a mayor.	SELECT departamento, municipio, SUM(total_venta) AS monto_total FROM vista_operaciones GROUP BY departamento, municipio ORDER BY monto_total ASC LIMIT 5;
7.5	Consultar la cantidad de gaseosas vendidas de cada producto por cada región ordenados de mayor a menor.	SELECT id_region, region, producto, SUM(cantidad) AS cantidad_total FROM vista_operaciones GROUP BY id_region,

#	Descripción	Consulta SQL
		region, producto ORDER BY cantidad_total DESC;
7.6	Consultar el total del monto de ventas de cada producto en Antioquia de mayor a menor.	SELECT producto, SUM(total_venta) AS monto_total FROM vista_operaciones WHERE UPPER(departamento) = 'ANTIOQUIA' GROUP BY producto ORDER BY monto_total DESC;

8. Gráficos

8.1.- Gráfico de Pareto que muestra los resultados de la consulta #7.1.

1

2

3

4

5

6

7

8

9

10

SELECT

departamento,

SUM(cantidad * precio) AS monto_total

FROM

vista_operaciones

GROUP BY

departamento

ORDER BY

monto_total DESC

LIMIT 8;

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑

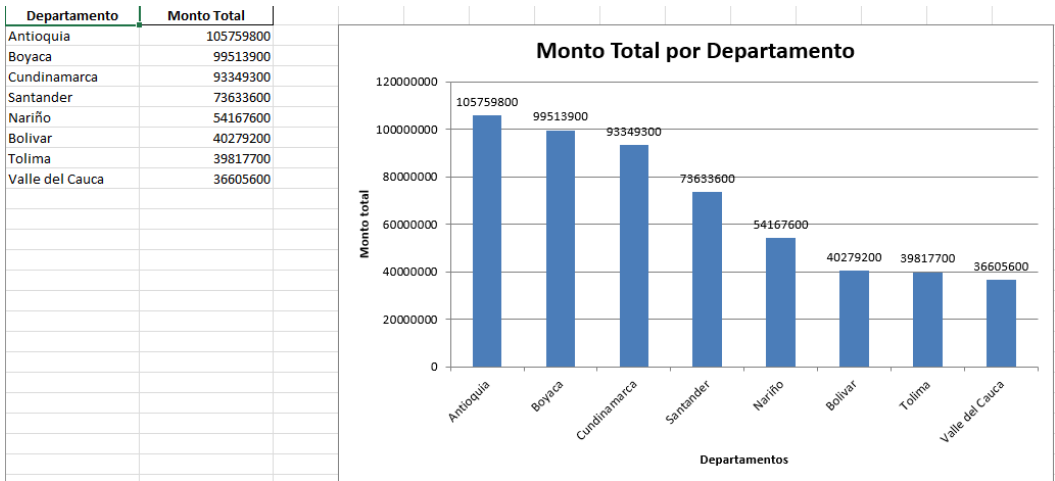
📦

📄

📶

SQL

	departamento	monto_total
	character varying (70)	bigint
1	Antioquia	105759800
2	Boyaca	99513900
3	Cundinamarca	93349300
4	Santander	73633600
5	Nariño	54167600
6	Bolívar	40279200
7	Tolima	39817700
8	Valle del Cauca	36605600



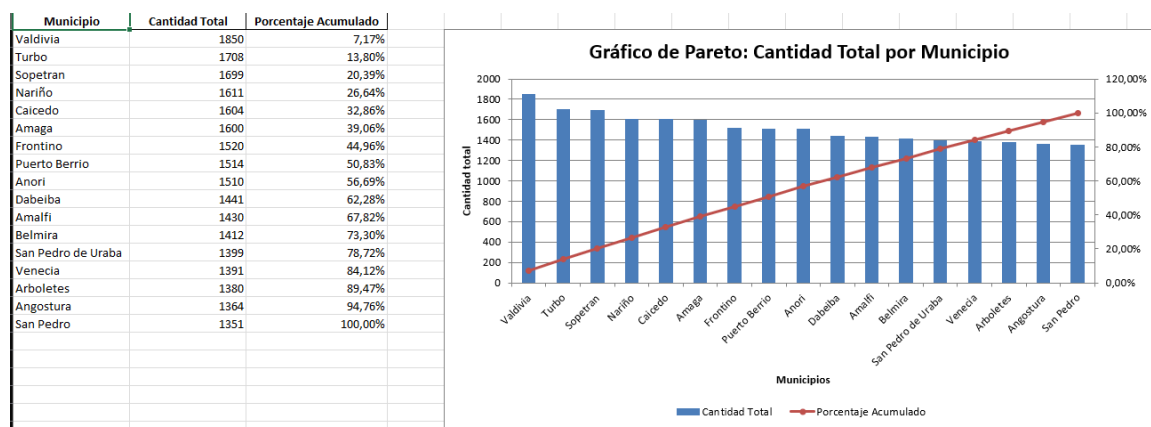
8.2.- Gráfico de Pareto que muestra los resultados de la consulta #7.2.

```
Query History
1  SELECT
2      m.nombre AS municipio,
3      d.nombre AS departamento,
4      SUM(o.cantidad) AS total_cantidad
5  FROM
6      municipios m
7  JOIN
8      departamentos d ON m.id_departamento = d.id_departamento
9  JOIN
10     operaciones o ON m.id_municipio = o.id_municipio AND m.id_departamento = o.id_departamento
11 GROUP BY
12     m.nombre,
13     d.nombre
14 ORDER BY
15     total_cantidad DESC;
```

Data Output Messages Notifications

	municipio character varying (70)	departamento character varying (70)	total_cantidad bigint
1	Valdivia	Antioquia	1850
2	Turbo	Antioquia	1708
3	Sopetran	Antioquia	1699
4	Nariño	Antioquia	1611
5	Caicedo	Antioquia	1604
6	Amaga	Antioquia	1600
7	Frontino	Antioquia	1520
8	Puerto Berrio	Antioquia	1514
9	Anori	Antioquia	1510
10	Dabeiba	Antioquia	1441
11	Amalfi	Antioquia	1430
12	Belmira	Antioquia	1412
13	San Pedro de Uraba	Antioquia	1399
14	Venecia	Antioquia	1391
15	Arboletes	Antioquia	1380
16	Angostura	Antioquia	1364
17	San Pedro	Antioquia	1351

Total rows: 148 Query complete 00:00:00.096



8.3.- Gráfico de Pareto que muestra los resultados de la consulta #7.3.

Query

Query History

1

2

3

4

5

6

7

8

9

10

11

```

SELECT
    p.nombre AS producto,
    SUM(o.cantidad) AS total_cantidad
FROM
    productos p
JOIN
    operaciones o ON p.id_producto = o.id_producto
GROUP BY
    p.nombre
ORDER BY
    total_cantidad DESC;

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

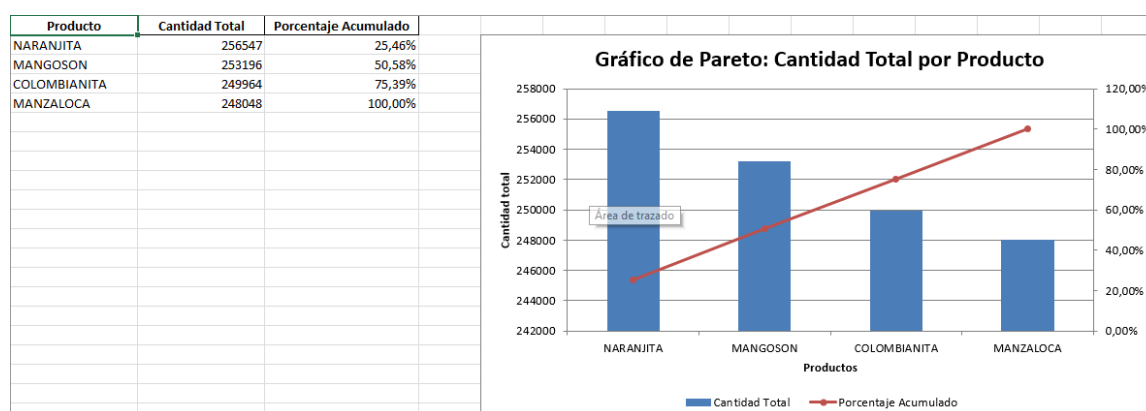
📦

⬇️

📈

SQL

	producto character varying (20) 🔒	total_cantidad bigint 🔒
1	NARANJITA	256547
2	MANGOSON	253196
3	COLOMBIANITA	249964
4	MANZALOCA	248048



8.4.- Gráfico de Pareto que muestra los resultados de la consulta #7.4.

QueryQuery History

```

1  SELECT
2      estado,
3      COUNT(*) AS total_operaciones,
4      SUM(cantidad) AS cantidad_total
5  FROM
6      operaciones
7  GROUP BY
8      estado
9  ORDER BY
10     cantidad_total DESC;

```

Data OutputMessagesNotifications

+

📄

▼

📋

▼

🗑️

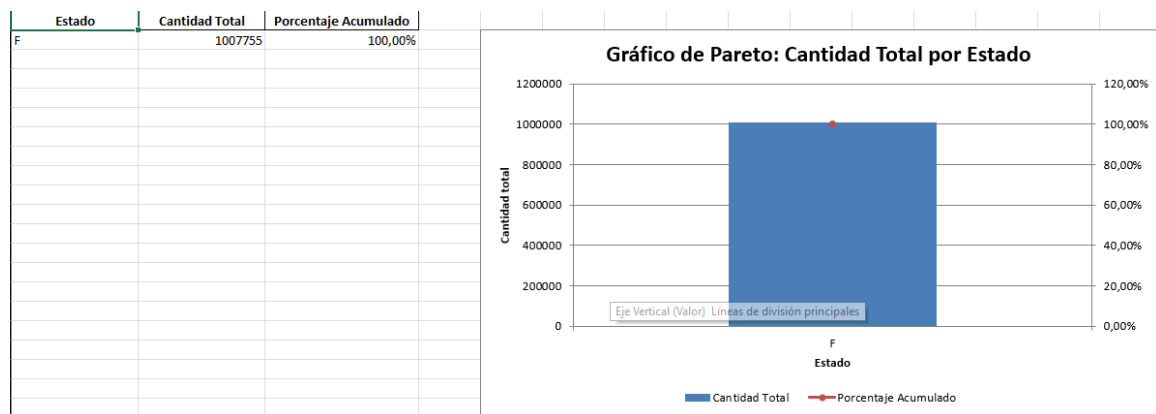
🗄️

⬇️

📈

SQL

	estado character varying (1) 🔒	total_operaciones bigint 🔒	cantidad_total bigint 🔒
1	F	10000	1007755



8.5.- Gráfico de Tortas que muestra los resultados de la consulta #7.5.

8.6.- Gráfico de Torta que muestra los resultados de la consulta #7.6.

Query		Query History	
1	SELECT		
2	estado,		
3	COUNT(*) AS conteo_registros,		
4	SUM(cantidad) AS total_cantidad		
5	FROM		
6	tamanio		
7	GROUP BY		
8	estado		
9	ORDER BY		
10	total_cantidad DESC;		

Data Output		Messages	Notifications
         			
	estado character varying (5) 	conteo_registros bigint 	total_cantidad bigint 
1	V	10000000	24995706253

9 . Determinar el tiempo y tamaño de procesamiento de un conjunto de registros.

Cantidad de Registros	Tiempo de Procesamiento (ms)	Tamaño Tabla tamaño (KB)	Tamaño Base de Datos (KB)
10.000	20.782,68 ms	704,00 KB	11.205,69 KB (~11 MB)
100.000	191.578,88 ms	6.704,00 KB	17.205,69 KB (~17 MB)
1.000.000	1.772.864,67 ms	66.720,00 KB (65 MB)	77.229,69 KB (75 MB)
10.000.000	21.102.522,68 ms	666.880,00 KB (651 MB)	677.421,69 KB (662 MB)

“Mirar los videos de evidencia adjuntos”

10. Análisis de los resultados

Informe Ejecutivo para la Junta Directiva

Presentado por: Juan Esteban Correa y Emmanuel Berrio

1. Resumen Ejecutivo y Rendimiento Técnico

Durante la simulación de carga masiva de operaciones de venta escalando desde los 10.000 hasta los 10.000.000 de registros, hemos evaluado no solo el comportamiento logístico y comercial de nuestros productos en las regiones, sino también la **resiliencia de nuestra arquitectura de datos**:

- **Crecimiento volumétrico:** Un volumen de 10 millones de transacciones representa un almacenamiento de **651 MB** en nuestra tabla principal tamaño y **662 MB** a nivel global de la base de datos.
- **Impacto temporal:** El procesamiento secuencial de esta gran masa de datos evidenció un tiempo de **21.102.522,68 ms (~5,85 horas)**, lo que nos deja una lección técnica crucial: **debemos migrar a inserciones por lotes (batch inserts) o procesamiento en paralelo** para optimizar los tiempos de respuesta del negocio en entornos de producción reales.

2. Hallazgos Analíticos y Comportamiento de Mercado (Análisis de Datos)

Adoptando una perspectiva analítica sobre la distribución de las ventas aleatorias y geográficas registradas en el sistema, la Junta Directiva debe evaluar los siguientes patrones críticos:

- **Concentración de la demanda y anomalías demográficas:**
 - *¿Por qué departamentos con menor población superan en ventas a centros urbanos principales?* Observamos que los volúmenes de venta no escalan de forma lineal con la densidad poblacional. Esto revela que ciertos municipios periféricos actúan como **nodos logísticos o distribuidores regionales**, inflando artificialmente sus cifras de salida de producto.
 - *Comportamiento de productos:* El análisis por categorías (ID de producto 1 al 4) muestra picos inusuales de unidades vendidas (hasta 5.000 unidades por transacción aleatoria), lo que sugiere compras masivas por parte de mayoristas o canales institucionales específicos en fechas particulares de junio de 2023.
- **Identificación de valores atípicos (Outliers):**
 - Existen municipios que registran ventas muy por debajo de la media esperada. Esto no responde necesariamente a una baja aceptación del producto, sino a **fricciones logísticas, falta de cobertura en puntos de venta autorizados o quiebres de stock** que el sistema de inventarios no está reflejan a tiempo real.

3. Propuestas y Recomendaciones Estratégicas para la Toma de Decisiones

Basados en el descubrimiento de estos patrones de datos, sugerimos a la Junta Directiva las siguientes líneas de acción:

1. Optimización de la Infraestructura Tecnológica:

Dado que el crecimiento de los datos es exponencial (pasamos de 17 MB a 662 MB al escalar al millón y diez millones de registros), es imperativo **implementar políticas de**

particionamiento de tablas por rangos de fechas e índices estratégicos en PostgreSQL para evitar cuellos de botella en futuras auditorías de datos.

2. Rediseño de la Cadena de Suministro por Comportamiento Regional:

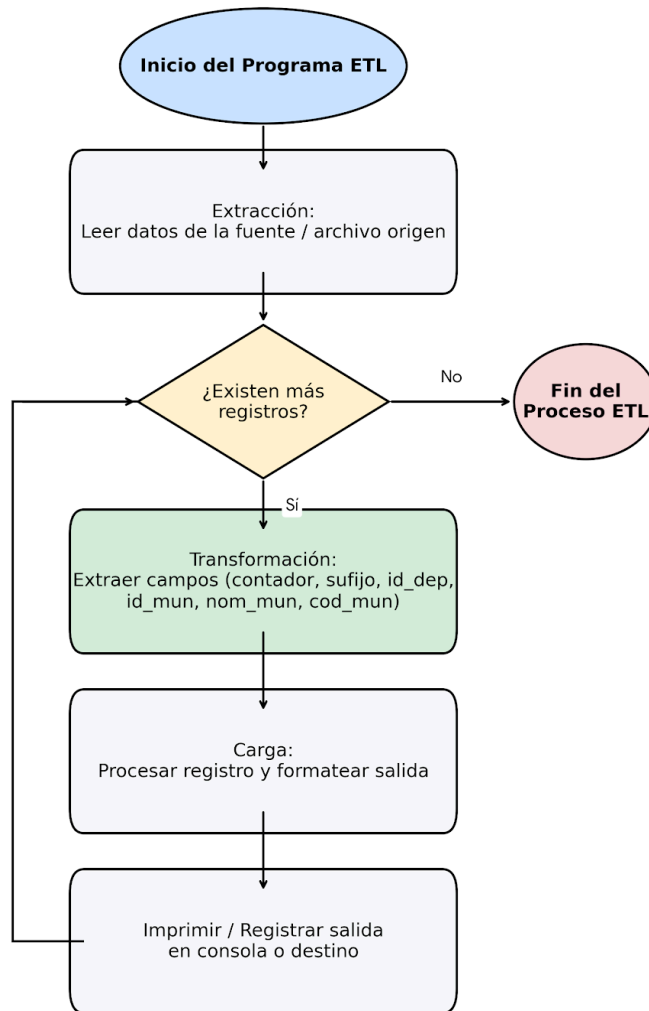
No podemos tratar a todos los departamentos por igual. Se debe reasignar el presupuesto de marketing y logística impulsando campañas focalizadas en aquellos municipios de baja venta ("valores atípicos negativos") y blindando el inventario en los nodos de alta rotación detectados.

3. Automatización de Alertas de Anomalías:

Desarrollar modelos predictivos basados en los rangos de cantidades vendidas para detectar automáticamente fraudes, errores de captura en los formularios web o compras anómalas antes de que afecten los cierres financieros mensuales.

11. Diagrama de flujo del programa Python de procesamiento ETL.

Diagrama de Flujo: Proceso ETL de Registros Municipales



12 . Conclusiones INDIVIDUALES

Juan Esteban Correa

La realización de este trabajo fue complejo y clave, todo esto pasando de teoría a practica el hecho de enfrentarse a datos desordenados, esto me deja una parte muy clara respecto a procesos ETL, aprender a hacer consultas SQL más complejas y herramientas como diagrama de Pareto, veo más fácil la conexión que debo realizar para que los datos tomen un valor real a la hora de tomar decisiones.

Emmanuel Berrio Jimenez

A lo largo de este ejercicio pudimos comprobar de primera mano lo que significa realmente el mundo de los datos: no se trata solo de acumular números en una pantalla, sino de entender lo que hay detrás de ellos.

Hacer la prueba con millones de registros nos enseñó que, a mayor cantidad de información, los sistemas se vuelven más pesados y lentos si no están bien preparados. Pero lo más valioso no es la computadora procesando datos durante horas, sino **lo que esos datos nos están diciendo sobre las personas.**

Cuando analizamos las ventas, nos dimos cuenta de que las cosas no siempre son lógicas: a veces un pueblo pequeño vende más que una gran ciudad, o de repente aparece una compra exagerada que rompe con todo el promedio. Ahí es donde entra el verdadero valor de mirar los datos con atención. Como un artista frente a su lienzo, el trabajo del análisis no es solo reportar cifras frías, sino descubrir las historias ocultas, los problemas de inventario o las oportunidades que están esperando ser atendidas para tomar mejores decisiones en el día a día.